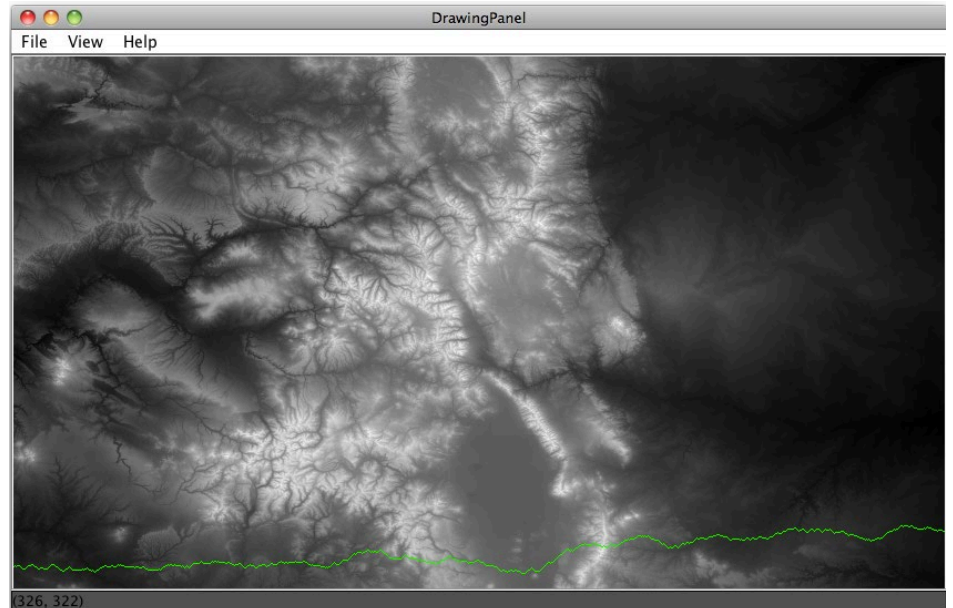


Programming Assignment: Mountain Paths

In this lab you will read a set of topographic (land elevation) data into a 2D array and write some methods to compute some paths through the mountains as well as visualize them.

Background:

There are many contexts in which you want to know the most efficient way to travel over land. When traveling through mountains (let's say you're walking) perhaps you want to take the route that requires the least total change in elevation with each step you take – call it the path of least resistance. Given some topographic data it should be possible to calculate a "greedy lowest-elevation-change walk" from one side of a map to the other.



In the map above, brighter shades mean higher elevation. The green line shows a lowest-elevation-change route for this map, traveling west-to-east.

A Greedy Walk

A "greedy" algorithm is one in which, in the face of too many possible choices, you make a choice that seems best at that moment. For the maps we are dealing with there are 7.24×10^{405} possible paths you could take starting from western side of the map and taking one step forward until you reach the eastern side.

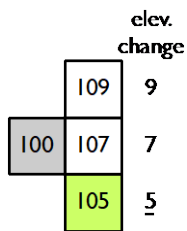
Since our map is in a 2D grid, we can envision a "walk" as starting in some cell at the left-most edge of the map (column 0) and proceeding forward by taking a "step" into one of the 3 adjacent cells in the next column over (column 1). Our "greedy walk" will assume that you will choose the cell whose elevation is closest to the elevation of the cell you're standing in. (NOTE: this might mean walking uphill or downhill).

3011	2900	2852	2808	2791	2818
2972	2937	2886	2860	2830	2748
2937	2959	2913	2864	2791	2742
2999	2888	2986	2910	2821	2754
2909	2816	2893	2997	2962	2798

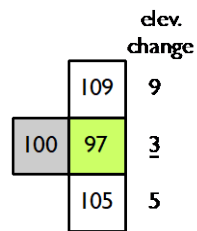
Shows a portion of the data.
Greedy path shown in green.

The diagrams below show a few scenarios for choosing where to take the next step. In the case of a tie with the forward position, you should always choose to go straight forward. In the case of a tie between the two non-forward locations, you should flip a

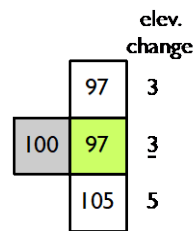
coin to choose where to go.



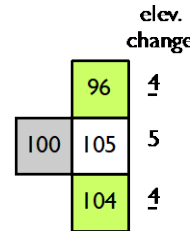
Case 1: smallest change is 5, go fwd-down



Case 2: smallest change is 3, go fwd



Case 3: smallest change is a tie (3), fwd is an option, so go fwd



Case 4: smallest change is a tie (4), choose randomly between fwd-up or fwd-down

There are other ways to choose a path through the mountains that can be explored in the "above and beyond" section of this assignment.

Assignment Requirements

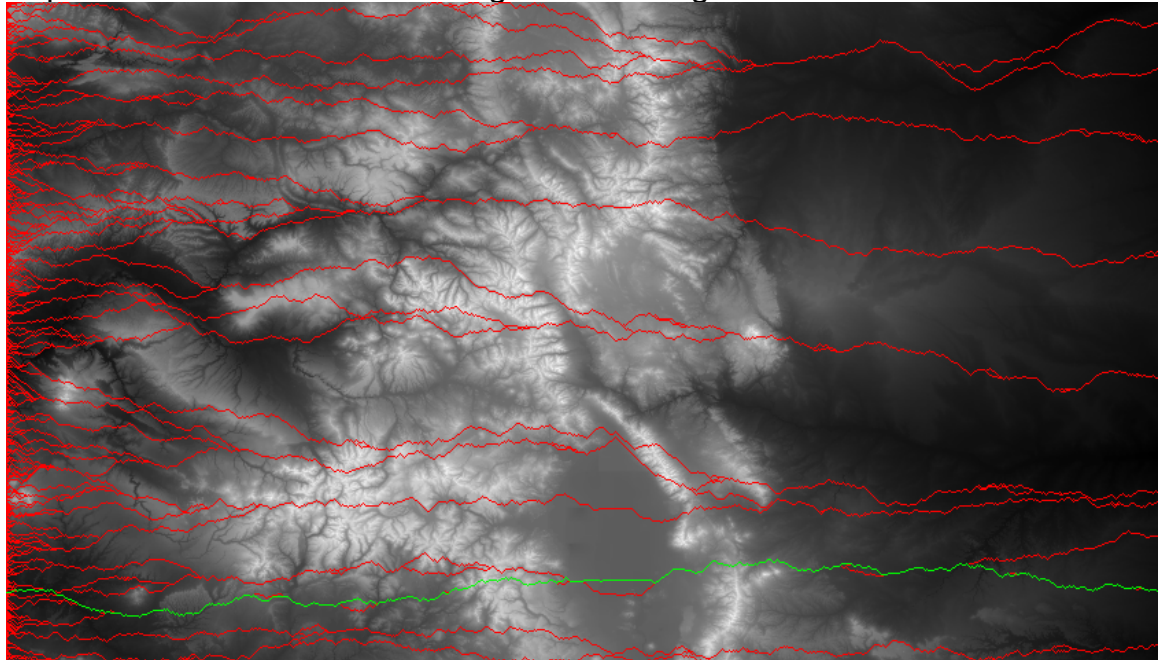
The minimum requirements for the assignment are that you write code to produce something like the map shown in the picture above. To do that you need to complete the following 5 steps:

Step 1: Read the data into a 2D array

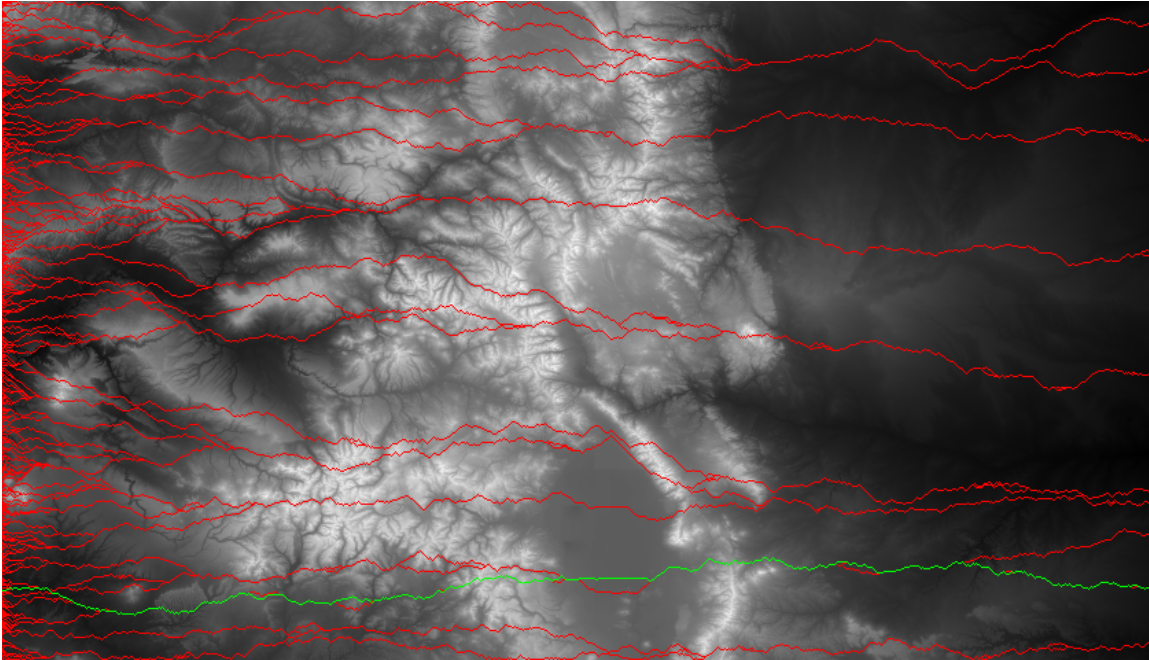
Step 2: Find min, max elevations, and other calculations on the data

Step 3: Draw the map

Step 4: Draw a lowest-elevation-change route starting from some row



Step 5: Find and draw the lowest-elevation-change route in the map



To fulfill the requirements you are provided with a class and method stubs for completing parts of the project. Here is the sketch:

```

class MapDataDrawer
    //a class for maintaining a 2D array of ints
    // representing a topographic map

    MapDataDrawer(String filename, int rows, int cols)
        //read data from given file into a 2D array

    int findMin()
        //return the minimum value in the map

    int findMax()
        //return the max value in the map

    void drawMap(Graphics g)
        //draw this map in B&W using given graphics context

    int indexOfMinRow(int col)
        //given a column, find the index of the row with min elevation
        // return row number of

    int drawLowestElevPath(Graphics g, int row)
        //draw the lowest elevation path starting from the given row
        // return total elev change from the path

    int indexOfLowestElevPath(Graphics g)
        //find the lowest elev change path in the map
        //return the row it starts on

```

Step 1 – Read the data into a 2D array

Your first goal is to read the values into a private 2D array of ints in the `MapDataDrawer` class. The constructor of the class should read the given data file into a new 2D array of ints maintained as a private variable of the class. To read in data from the file, you need to use a try-catch exception to deal with `FileNotFoundException`. I will explain in class. If you are reading this and I haven't explained it, please ask so I can explain to everyone.

The data

The data is a plain ascii text file from NOAA representing the average elevations of patches of land (roughly 700x700 meters) in the US.¹ The data file included with the project, [colorado.dat](#), is the elevation data for most of the state of Colorado (mountains!). The data comes as one large, space-separated list of integers. There are 403,200 integers representing a 480-row by 844-col grid. Each integer is the average elevation in meters of each cell in the grid.

In the constructor:

- Instantiate a new 2D array of ints of the appropriate size
- Read the ints out of the data file (the data are listed in row-major order).
- WARNING: The data are given as a continuous stream of numbers - there are no line breaks in the file, so you can't use `nextLine()`. You need to use Scanner's `nextInt()` method to read each subsequent in. You also probably want to write nested loops to cover the exact rows/cols (480/844) needed.

After you've written the constructor, **test it on the object bench, and/or with the debugger**. Make sure that you're actually reading data into the 2D array. Do a sanity check by looking at a specific row and col and comparing with a friend, or against the original data itself.

Step 2 – Find the min and max values

In order to draw the map you need to find the min and max values in the map first. Implement the **`findMin()`** and **`findMax()`** methods. These should return the smallest and largest values respectively in the 2D array. You'll need to write code that scans the entire array and keeps track of the smallest and largest things found so far.

Test these functions independently first to make sure you're getting decent values. Maybe check with a friend to see if s/he is getting the same thing.

Step 3 – Draw the map

Implement the **`drawMap(Graphics)`** method of the class. The method will be passed the `Graphics` object you should use to do the drawing.

¹ <http://maps.ngdc.noaa.gov/viewers/wcs-client/>

The method should “draws” the 2D array of ints with the given Graphics object as a series filled 1x1 rectangles with the color set to a grayscale value between white and black.

The shade of gray should be scaled to the elevation of the map. Recall, in RGB color, if each of RGB values are the same, you’ll get a shade of gray. Thus, there are 256 possible shades of gray from black (0,0,0) to middle gray (128,128,128), to white (255,255,255)

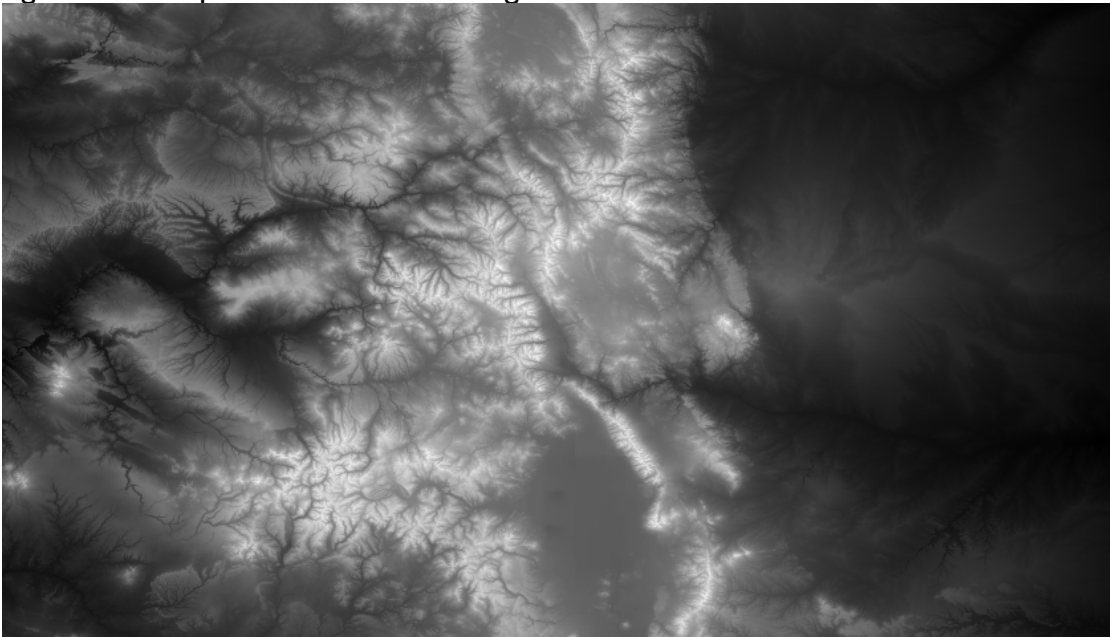
To make the shade of gray, you should use the min and max values in the 2D array (you just wrote helper methods to find these!) to scale each int to a value between 0 and 255 inclusive. (**Note:** this requires math! Think about it.) Once you have found your value between 0 and 255, call you need to set the fill color just before drawing the rectangle.

```
int c = //calculated grayscale value
g.setColor(new Color(c, c, c));
g.fillRect(x,y,1,1)
```

WARNING: the 2D array represents the data in row-major order (that’s how it came out of the data file). But the graphics are drawn in column-major order x,y format. Thus, remember to flip row and col (or x and y) if using them as variables to access the array or draw the graphics

```
grid[row][col] → g.fillRect(col,row,1,1)
or
grid[y][x] → g.fillRect(x,y,1,1)
```

This step is easier to test because there is graphical output to verify you’re doing it right. Your map should look something like this:



Step 4 – Draw a lowest-elevation-change path from some row

Implement the `drawLowestElevPath(Graphics, startRow)` method. Note that this method does two things: 1) it draws the path, 2) it calculates and returns to the total elevation change on that path.

The method should draw a line by drawing 1x1 filled rects in a different color on top of the existing drawing. The path should be drawn going West-to-East, starting from the given row using the greedy lowest-elevation-change technique described in earlier pages.

You will need to do the following things in some order.

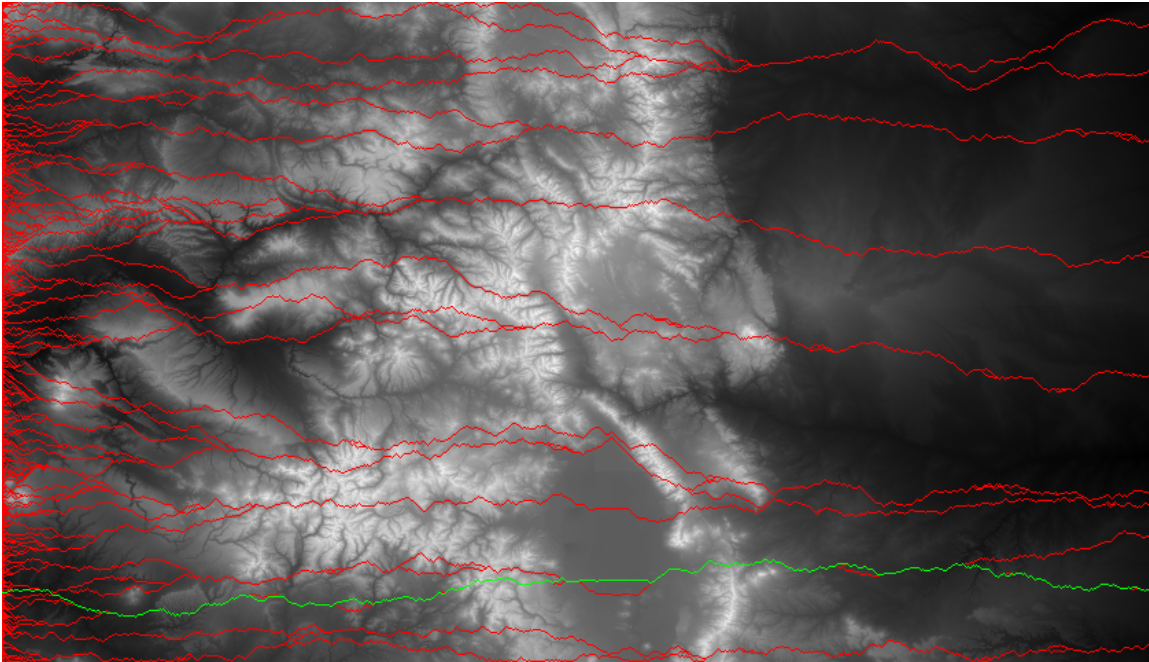
1. Starting from the given row, and column 0, color it green (or whatever color you want to draw your path).
2. Write a loop that generates every possible column across the map from 1 to 840 (you start at column 0, but column 1 is the first column you need to make choice about where to step). For each column you will decide which row to take your next “step” to – fwd, fwd-and-up, or fwd-and-down - using the greedy choice strategy described.
3. Color the chosen location to step to green (or whatever color you choose).
4. Use a variable that keeps track of the ‘current row’ you’re on, and update it each time you take a step forward – row may stay the same, or go up or down by 1 depending how you walk.
5. Use `Math.abs(...)` to get the absolute value of the difference between two elevations.
6. Continue finding the lowest neighboring cell and coloring it green as you go.
7. Keep a running total of the total elevation change that would be ‘experienced’ by a person walking this path. Since we consider an elevation change the absolute value (i.e. going ‘uphill’ 10 meters is the same amount of change as going ‘downhill’ 10 meters) this running total will be non-decreasing and will end up being a pretty large positive number

When you’re done you should see a line tracing the lowest elevation-change path from west to east.

Step 5 – find the lowest elevation path for the whole map

Implement the `indexOfLowestElevPath()` method which finds the overall lowest-elevation-change path and returns the row it starts on.

You will find the lowest-elevation-change route by noting that the method you wrote in step 4 returns the total elevation change for a path starting at a given row. If you write a loop that generates every possible starting row from 0 to 480, and call your method using each possible starting row, you should find the lowest route for the whole map. (Since that method also draws all of the routes it's good feedback that the method is running properly.) You might end up with something that looks like this:



KLUGE NOTE: Even though the `indexOfLowestPath` method accepts a `Graphics` object, this method should not use it to draw anything, but rather simply pass it onto `drawLowestElevPath` method to that it can use it to draw. (This is possibly a flaw in the program design, but a flaw designed to protect you from some complicated `Object` coupling that we could avoid.)

If you want to, you can clear the map after this calculation and simply draw the one line. The result should look *something* close to the image above or the one shown on the first page of the assignment.

Sanity Check:

NOTE: The greedy walk should always produce a *slightly* different path due to the fact that you are flipping a coin in some cases to determine which direction to go.

The best greedy walk for the `Colorado.dat` map should return a total elevation change somewhere around 16,000 +/- 1000 (meters) (sometimes this will jump to around 21000 depending on the run). The best path should start somewhere close to the southern edge of the map, something between row around 420-450.

Please submit your `MapDataDrawer.java` (45 pts) and a txt file describing your struggles and problem solving breakthroughs (5 pts).